



B.TECH COMPUTER SCIENCE & ENGINEERING

1st Year | 2nd Semester | Python Programming

PROJECT REPORT ON TRAFFIC DATA ANALYSIS SYSTEM

Submitted By	Kamal Khandelwal
SAP ID	590021614
Course	B.Tech CSE – 1st Year
Semester	2nd Semester
Subject	Python Programming
Academic Year	2025-2029

Project Pipeline

This is the complete data flow of the system — from input to final output.

Step	Stage	Details
1	Data Input	CSV File / Manual Entry
2	Data Loading	load_data() → List of Dicts
3	Processing	Filter / Search / Validate
4	Analysis	NumPy: mean, std, max, min
5	Visualization	Matplotlib: Bar / Line Chart
6	Output	Console Display / File Append

1. Project Synopsis

The Traffic Data Analysis System is a Python-based console application. It reads traffic records from a CSV file and allows the user to filter, search, add, analyze, and visualize data using NumPy and Matplotlib.

2. Project Proposal

This project proposes building a menu-driven traffic data management tool in Python. It will use file handling to store data in CSV format, NumPy for statistical calculations, and Matplotlib for graphical output.

3. Project Overview

Main Goal

To build a complete traffic data system that lets a user manage, analyze, and visualize road traffic records through a simple command-line menu.

Key Modules

- Summary Viewer — shows total records, unique locations and days
- Data Filter — filter by day or location
- Search — keyword-based record lookup
- Add Record — input validation + append to CSV
- NumPy Analysis — statistical summary of vehicle count and speed
- Matplotlib Graphs — bar and line charts
- Unique Values — set-based display with missing days

4. Problem Statement

Manual traffic data tracking is slow, error-prone, and difficult to analyze at scale. There is a need for an automated system that:

- Stores traffic records in a structured file format
- Allows filtering and searching by day or location
- Calculates statistical summaries (mean, std, max, min)
- Visualizes vehicle count and speed trends using graphs
- Handles all edge cases through proper exception handling

5. Project Objectives

-
- Store and retrieve traffic data using CSV file handling
 - Implement menu-driven navigation using while loop and conditionals
 - Use class TrafficRecord with methods show() and summary()
 - Perform NumPy-based statistical analysis on numeric fields
 - Display bar and line charts using Matplotlib
 - Use sets to find unique locations/days and detect missing data
 - Handle all input errors with try/except blocks

6. Methodology

Phase 1 – Design

Defined the TrafficRecord class structure, planned the CSV format, and outlined the main menu flow.

Phase 2 – Implementation

Built all 7 feature functions (view_summary, filter_data, search_records, add_record, analyze_traffic, show_graphs, show_unique) .

Phase 3 – Testing

Tested with sample data (12 records, 4 locations, varied days). Validated all edge cases: invalid input, missing file, negative values.

Tools Used

- Language: Python 3
- Libraries: NumPy, Matplotlib, csv, os
- Storage: CSV flat file (traffic_data.csv)
- IDE: Any Python-compatible editor

7. Implementation Details — Python Concepts

Below is a brief summary of each:

Concept	Where Used in Code
Lists	<code>records = []</code> stores all CSV rows; <code>rows = [...]</code> in <code>create_sample_file()</code>
Tuples	<code>DAYS_OF_WEEK = (...)</code> — fixed, used for validation and display
Dictionary	<code>csv.DictReader</code> returns dicts; <code>d = {}</code> used in graph aggregation
Loops	<code>for r in records</code> in every function; <code>while True</code> for main menu
Conditions	<code>if/elif/else</code> for menu choices, input validation, empty checks
Strings	<code>.strip()</code> , <code>.lower()</code> , <code>.title()</code> , <code>.join()</code> , f-strings throughout
Functions	Each feature is a separate <code>def</code> function for modularity
File Handling	<code>open()</code> with 'w', 'r', 'a' modes; <code>csv.writer</code> and <code>csv.DictWriter</code>
Exception Handling	<code>try/except</code> in <code>load_data()</code> , <code>add_record()</code> , <code>analyze_traffic()</code> , <code>show_graphs()</code>
NumPy	<code>np.array()</code> , <code>np.mean()</code> , <code>np.sum()</code> , <code>np.max()</code> , <code>np.min()</code> , <code>np.std()</code> , <code>np.median()</code>
Matplotlib	<code>plt.bar()</code> , <code>plt.plot()</code> , <code>plt.title()</code> , <code>plt.xlabel()</code> , <code>plt.show()</code>
Sets	<code>set()</code> removes duplicate locations/days; <code>set</code> difference for missing days
Classes & Objects	<code>class TrafficRecord</code> with <code>__init__</code> , <code>show()</code> , <code>summary()</code> ; objects created in 4 places

8. Deliverables

- Python source file: traffic_analysis_final.py
- CSV data file: traffic_data.csv (auto-generated if missing)
- Console output: menu, filtered tables, stats
- Graphs: bar chart (vehicles by location/day), line chart (speed)
- Project report document (this file)

9. Conclusion

The project is well-structured, modular, and easy to extend. It covers data storage (file handling), data processing (loops, conditions, functions), and features (classes, sets, exception handling), and data analysis and visualization (NumPy, Matplotlib). This project helped in understanding how Python can be used to build complete, functional applications from scratch.